PARK IT: DATABASE MANAGEMENT SYSTEM

DETAILED PROJECT AND ACTIVITY PROGRESS DOCUMENTATION

Group Number: 2

Course / Section: COMP 018 – Database Administration / BSIT 3-1

Instructor: John Kenneth Timblaco

Documentation Coverage: October 25 – November 6, 2026

GROUP MEMBERS

Member	Assigned Role
De Vera Miller P.	Project Manager
Butanas John David	UI/UX
Macascas Mark Andrew	Front-end
Pelia Jhon Francis	Back-end
Lumbo Marianne	Quality Assurance

Project Repository: https://github.com/HelloP0/PARK-IT-DATABASE-MANAGEMENT-SYSTEM.git

Project Tracker: [BSIT 3-1] – PROJECT TRACKER.xlsx – Google Sheets

1. INTRODUCTION AND PROJECT BACKGROUND

1.1 Project Introduction

Park It: Database Management System is a parking-management application intended to organize the records and daily activities involved in operating a parking facility. The project addresses common problems associated with manual parking records, including delays in recording vehicle entry and exit, difficulty checking available spaces, and mistakes in calculating or recording payments.

The application brings these activities into one system. It is designed to record vehicle information and entry time, track active parking transactions, calculate parking fees based on parking duration, display parking-space availability, and keep payment information. The database serves as the central location for storing the records used by the application.

1.2 Project Objectives and Scope

The main objective is to provide an organized way to manage parking operations and reduce dependence on manual recording. The system scope covers vehicle registration, entry and exit monitoring, parking transactions, parking-zone capacity, payment records, and staff accounts. It also includes the database operations needed to create, retrieve, update, and delete records.

1.3 Main Application Roles

The original project documentation identifies three application roles. The Administrator manages users, parking records, and system operations. The Parking Attendant handles vehicle entries, exits, and transactions required for daily operations. The Vehicle Owner uses the parking service and receives relevant parking and payment information. These are application-level roles; their exact database privileges must be defined separately through the access-control work.

Application role	Main responsibility
Administrator	Manages users, parking records, and system operations.
Parking Attendant	Handles vehicle entries, exits, and parking transactions.
Vehicle Owner	Uses the parking service and views relevant parking and payment information.

2. DATABASE AND DEVELOPMENT ENVIRONMENT

The project uses PostgreSQL as its relational database management system. The database name recorded in the original project documentation is grp_parkitdb. The database structure contains five documented tables. The application development environment also uses Node.js and Git, together with a code editor or integrated development environment.

Database object	Purpose recorded in the project documentation
parking_zone	Stores parking-zone details and capacity. Primary key: zone_id.
vehicle	Stores vehicle details, including license plate and vehicle type. Primary key: vehicle_id.
transaction	Records vehicle parking transactions, including entry time, exit time, and status. Primary key: transaction_id.
payment	Stores payment details associated with parking transactions. Primary key: payment_id.

staff_user	Stores staff account information used by the system. Primary key: user_id.

The ERD and SQL schema image in the original report show the planned relationships and table definitions. The five tables work together: vehicle details are associated with parking activity, parking activity records entry and exit information, and payment records are connected to the corresponding transaction. The parking_zone table supports capacity monitoring, while staff_user stores staff account information.

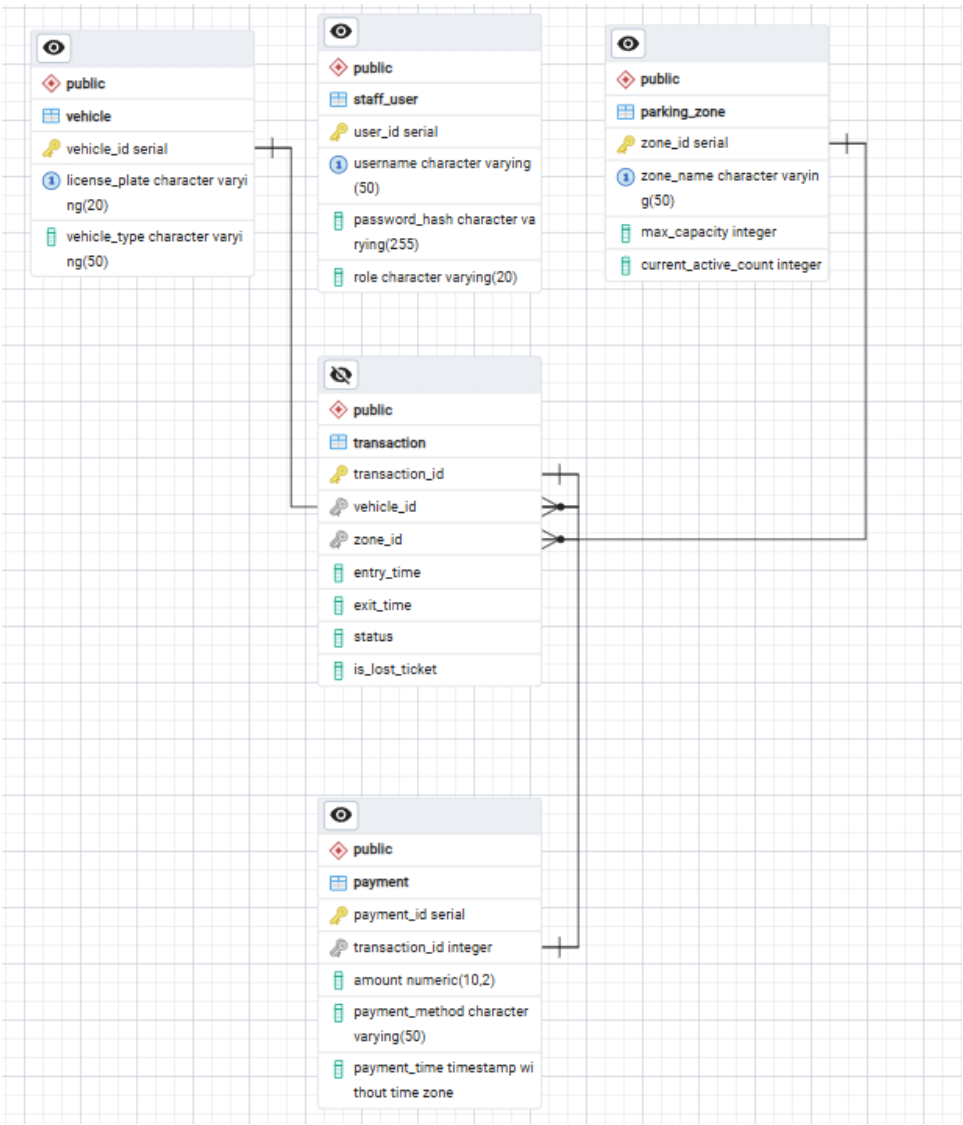


Figure 1. Entity Relationship Diagram of the Park It database.

```sql
CREATE TABLE public.parking_zone (
    zone_id integer NOT NULL,
    zone_name character varying(50) NOT NULL,
    max_capacity integer NOT NULL,
    current_active_count integer DEFAULT 0 NOT NULL,
    CONSTRAINT parking_zone_check CHECK ((current_active_count <= max_capacity))
);

CREATE TABLE public.payment (
    payment_id integer NOT NULL,
    transaction_id integer NOT NULL,
    amount numeric(10,2) NOT NULL,
    payment_method character varying(50) NOT NULL,
    payment_time timestamp without time zone NOT NULL,
    CONSTRAINT payment_amount_check CHECK ((amount >= (0)::numeric))
);


CREATE TABLE public.staff_user (
    user_id integer NOT NULL,
    username character varying(50) NOT NULL,
    password_hash character varying(255) NOT NULL,
    role character varying(20) NOT NULL
);


CREATE TABLE public.transaction (
    transaction_id integer NOT NULL,
    vehicle_id integer NOT NULL,
    zone_id integer NOT NULL,
    entry_time timestamp without time zone NOT NULL,
    exit_time timestamp without time zone,
    status character varying(20) DEFAULT 'Active'::character varying NOT NULL,
    is_lost_ticket boolean DEFAULT false NOT NULL
);

CREATE TABLE public.vehicle (
    vehicle_id integer NOT NULL,
    license_plate character varying(20) NOT NULL,
    vehicle_type character varying(50) NOT NULL
);
```

Figure 2. SQL table definitions shown in the original project documentation.

3. CHRONOLOGICAL PROGRESS OVERVIEW

The documentation covers four consecutive course activities. Activity 3 focused on preparing and documenting the Park It application and its database. Activity 4 extended the database-administration work to access-control planning and privilege testing for assigned class projects. Activity 5 followed by checking database health, reviewing available logs, performing suitable maintenance, and examining query execution plans. Activity 6 then focused on protecting database contents through logical backups, restoration to a separate test database, and a controlled recovery exercise.

Activity	Reporting period	Main work
Activity 3	October 25–27, 2026	Park It system implementation, database structure, application evidence, and repository documentation.
Activity 4	October 27–30, 2026	Database setup, role planning, privilege assignment, and allowed/denied access testing for assigned projects.
Activity 5	October 30–November 2, 2026	Database health checks, session and log review, maintenance, and query-plan analysis.
Activity 6	November 2–6, 2026	Backup planning, logical backup creation, archive inspection, test restoration, and recovery evaluation.

4. ACTIVITY 3: SYSTEM DEVELOPMENT AND REPOSITORY DEPLOYMENT

Reporting period: October 25–27, 2026

4.1 Reviewing the system requirements and assigning responsibilities

The group began by reviewing the purpose and scope of Park It and identifying the information the application needed to store. The team listed the main parking activities that the database had to support: vehicle registration, recording vehicle entry and exit, tracking parking transactions, monitoring available spaces, and recording payments. This helped the group identify which data should be stored and which application screens would be needed.

Responsibilities were divided according to the members' assigned roles. De Vera Miller P. handled project management and was listed as responsible for the initial planning, requirements gathering, database design, ERD, database creation, and database connection work. Butanas John David and Macascas Mark Andrew worked on interface improvements. Macascas Mark Andrew and Pelia Jhon Francis were assigned CRUD functionality development. Lumbo Marianne handled system testing and debugging.

4.2 Designing the database and preparing the ERD

After identifying the required information, the group organized the data into separate tables instead of placing all information in one large record. The five documented tables were parking_zone, vehicle, transaction, payment, and staff_user. Each table was given a primary key so that every record could be identified. The ERD was prepared to show how the tables relate to one another, particularly how a vehicle is connected to its parking transaction and how a payment is associated with a transaction.

The group used the ERD as a reference when preparing the SQL structure. This step was important because the application depends on the database relationships to retrieve related information consistently. The schema screenshot retained from the original report shows the CREATE TABLE statements and the fields and constraints defined for the database.

4.3 Creating the PostgreSQL database and tables

The original documentation identifies the PostgreSQL database as grp_parkitdb and records that five tables were created. The database setup translated the ERD into database objects. The SQL definitions created the tables and specified their columns, data types, primary keys, foreign-key relationships, and selected constraints. For example, the parking_zone structure includes capacity-related fields and a check intended to prevent the current active count from exceeding the maximum capacity. The payment structure stores the transaction reference, payment amount, method, and payment time.

The original report also records that the database design, ERD development, and database creation were marked 100% complete. The retained ERD and SQL schema images are included above as the supporting evidence from the earlier documentation.

4.4 Connecting the application to the database

Once the database structure was prepared, the group worked on connecting the application to PostgreSQL. The original development log lists De Vera Miller P. and Pelia Jhon Francis as responsible for database connection implementation. The project setup instructions identify the required environment as Node.js, Git, PostgreSQL, and an IDE. They also describe configuring the database connection through environment variables such as DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, and DB_PORT. These variable names are configuration placeholders; actual credentials should remain private and must not be written in the report or committed to the repository.

The purpose of the connection step was to allow the application to send database requests and display stored records. The group needed to confirm that the application could reach the PostgreSQL server, use the correct database name, and access the required tables. The project documentation records the

database connection work as completed at 80% in the progress summary, while the development log labels the connection implementation as completed. These two status descriptions are retained as they appeared in the original report and should be reconciled against the group's dated commits or test records before final submission.

4.5 Building the user interface

The original screenshots show a login screen and several application modules. The login screen provides the entry point to the system. After signing in, the dashboard presents a summary of parking activity, including total parking capacity, occupied spaces, available spaces, and recent active parking sessions. This dashboard is intended to give an administrator or attendant a quick view of the current parking situation.



Figure 3. Login interface of the Park It application.

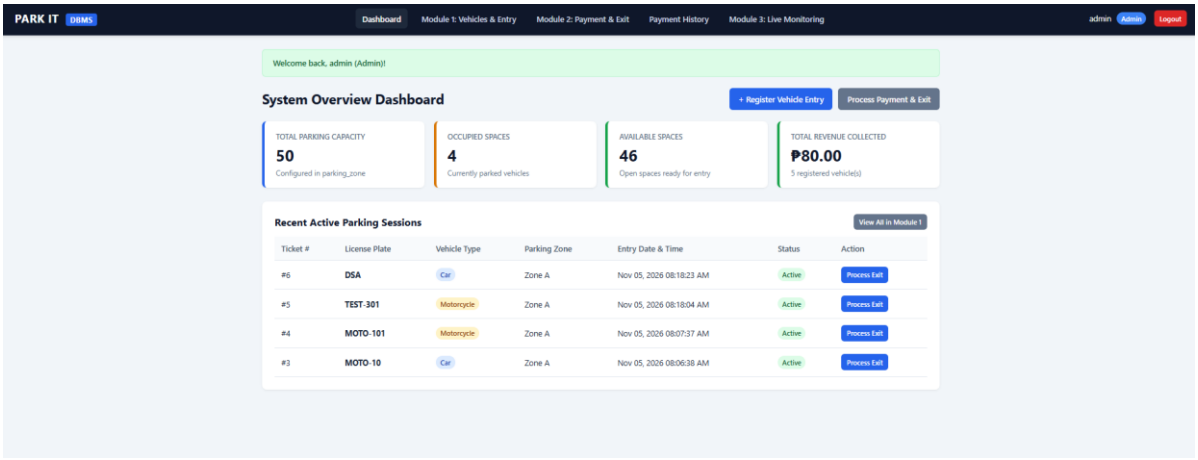


Figure 4. Main dashboard displaying parking information.

The vehicle-management module provides a form for registering an incoming vehicle and a list of active parking transactions and registered vehicle records. This supports the recording of a vehicle's license plate, vehicle type, and assigned parking zone. The interface also provides access to the existing records so that staff can manage the vehicle information.

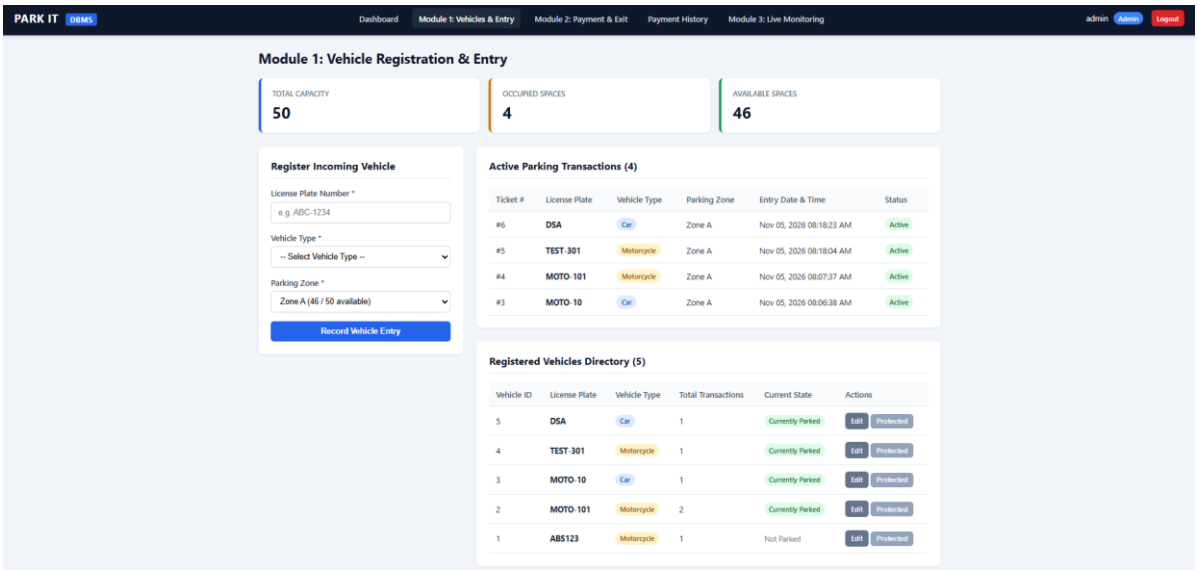


Figure 5. Vehicle registration and record-management interface.

The parking-monitoring module displays parking capacity and occupancy information. It presents parking-zone details and active parked vehicles so that users can check which spaces are occupied and how many spaces remain available. This screen connects the application's visual status information to the data stored in the parking_zone and transaction records.

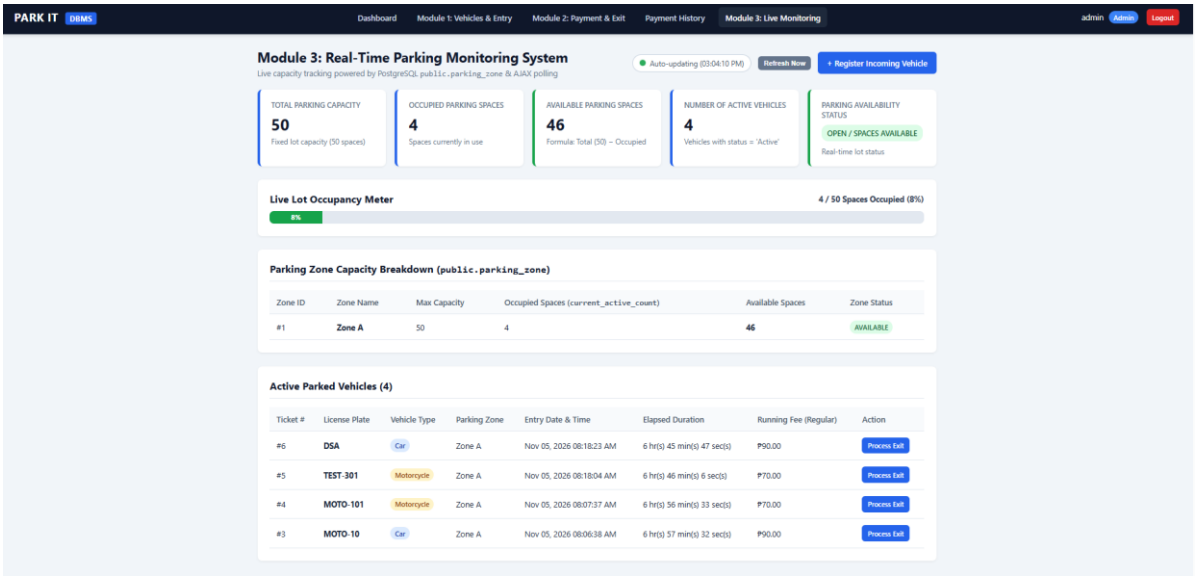


Figure 6. Parking-space monitoring interface.

The payment-management module displays payment and completed-transaction information. It provides a view of payment records and the information associated with a parking transaction. This supports checking whether a parking session has been paid and recording the relevant payment details.

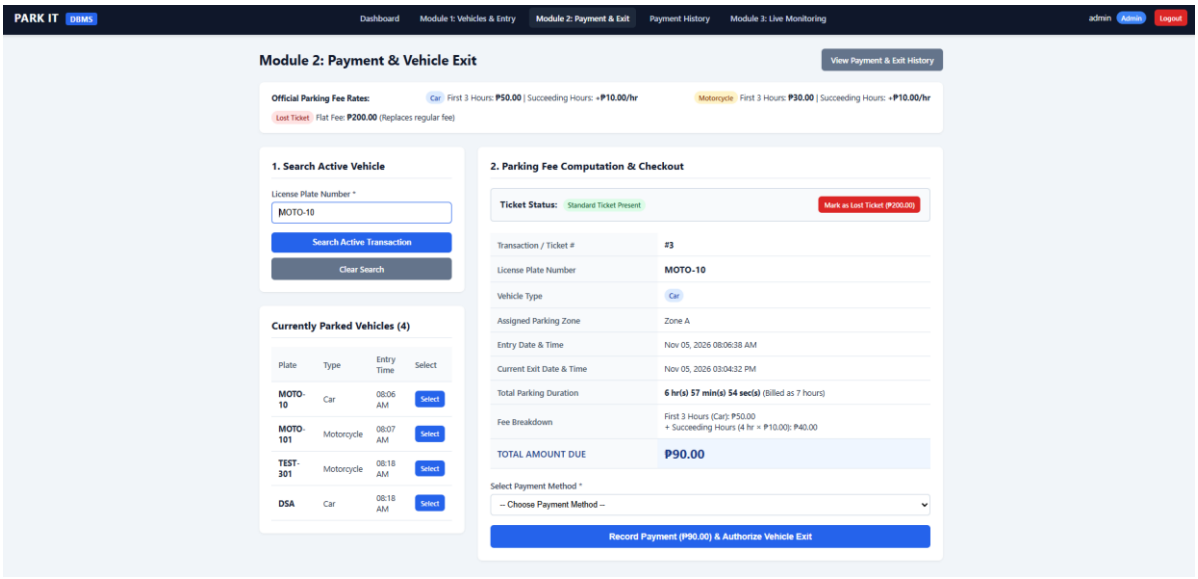


Figure 7. Payment information and verification interface.

4.6 Implementing and checking CRUD operations

CRUD refers to Create, Retrieve, Update, and Delete. These operations are necessary because the system must do more than display a static interface. It must save new vehicle or parking records, retrieve existing information, update a record when its details change, and remove a selected record when deletion is appropriate.

The original report includes four CRUD test categories. The Create test checks whether a new record can be submitted and saved. The Retrieve test checks whether existing records can be displayed from the database. The Update test checks whether an existing record can be changed and whether the updated information appears afterward. The Delete test checks whether a selected record can be removed. The original report marks all four tests as completed and includes screenshots for each test.

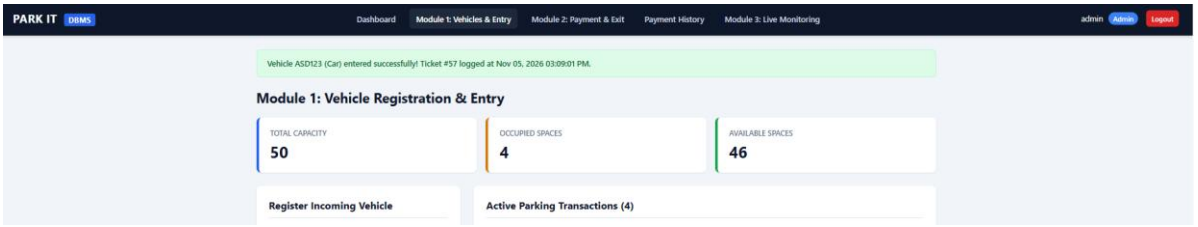


Figure 8. Create test evidence retained from the original report.

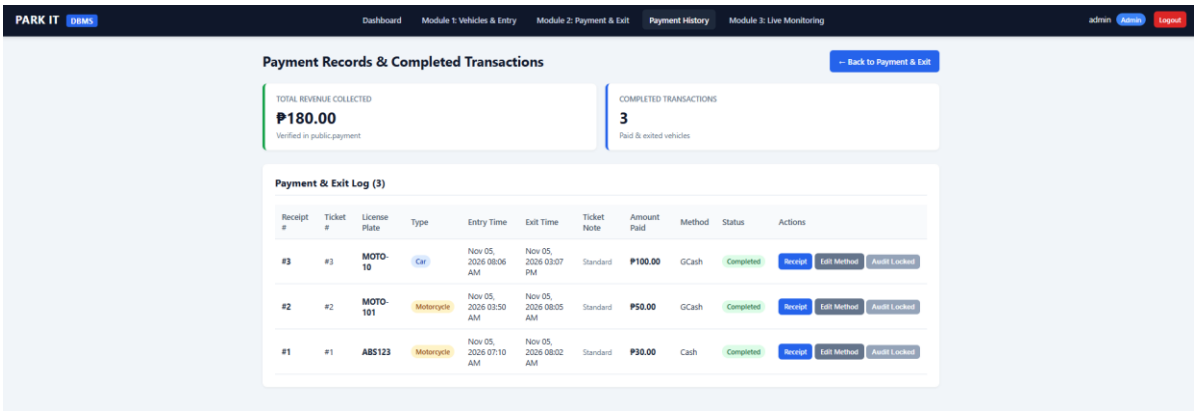


Figure 9. Retrieve test evidence retained from the original report.

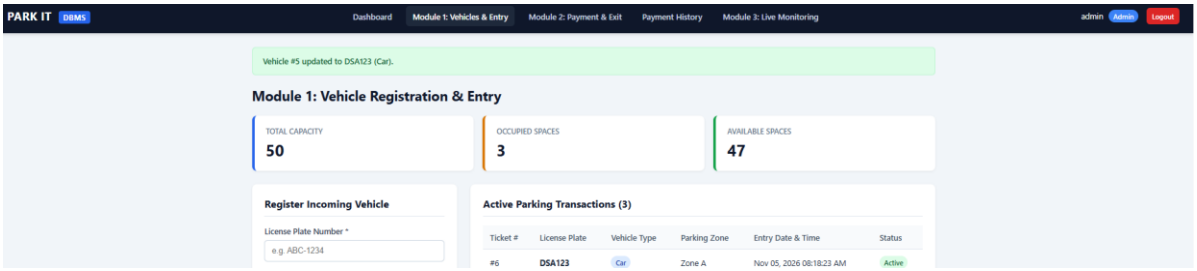


Figure 10. Update test evidence retained from the original report.

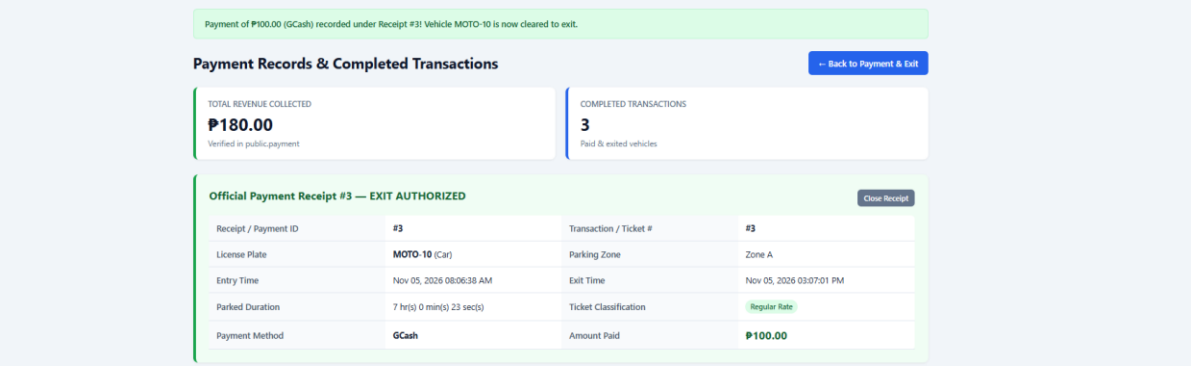


Figure 11. Delete test evidence retained from the original report.

The original development progress table lists CRUD implementation as ongoing at 60%, while the CRUD testing section marks Create, Retrieve, Update, and Delete as completed. These statements may refer to different stages of implementation and testing, but the original report does not explain the difference. The group should update the progress percentage using the latest dated test evidence rather than assuming that the two statements describe the same measurement.

4.7 Repository organization and documentation

The group used its own GitHub repository to keep the project files and track changes. The original report identifies key commit areas as initial project files, database connection implementation, CRUD functionality updates, and interface improvements. The repository also serves as the location for setup instructions and database-related resources. The installation guide describes installing the required tools, cloning the repository, opening the project folder, installing dependencies, configuring the database connection, creating the database, inserting seed data, starting the application, and opening it in a browser.

The issues recorded in the original report included an initially static interface, Node.js and Git commands not being recognized because of path configuration, and Git merge conflicts during a final push. The documented responses were to implement CRUD functionality, check and correct installation paths before restarting the development environment, and resolve the merge conflict by updating the main branch from the working local copy.

4.8 Activity 3 outcome and transition

Activity 3 established the Park It project's documented application structure, database name, five-table schema, intended user roles, interface evidence, and CRUD test evidence. This information became the reference for later database-administration activities. In particular, the project's user roles and data objects helped the group understand why database accounts should receive only the permissions required for their tasks.

5. ACTIVITY 4: DATABASE ACCESS CONTROL

Reporting period: October 27–30, 2026

5.1 Purpose and preparation

Activity 4 moved from application development to database security. The task required the group to work with assigned classmates' projects using separate PostgreSQL databases and to document how access should be controlled. The group first identified the database name used by each assigned project, reviewed the project's tables and other database objects, and checked the application roles described by the project owner. This preparation was necessary because role names and permissions must match the actual project rather than being copied without checking.

The group then prepared an access-control plan. The plan connected each database login account to a group role and an application responsibility. Typical role categories in the activity instructions included an administrator role, an application-runtime role, and a read-only or auditor role. A data-entry role could also be used when the application required a user to insert and view records. The exact role names and privileges were supposed to come from each project's own plan and SQL scripts.

5.2 Creating group roles and login accounts

The SQL work was separated into manageable scripts so that role creation and privilege assignment could be checked independently. The first part created non-login group roles to represent permission sets. Login roles were then created for the accounts that would connect to PostgreSQL. Membership statements connected each login account to the appropriate group role. This structure makes it easier to change permissions centrally without repeating grants for every individual account.

```
-- Example structure only; use the role names from the project's plan.
CREATE ROLE app_runtime_grp NOLOGIN;
CREATE ROLE app_readonly_grp NOLOGIN;

CREATE ROLE app_runtime_user LOGIN;
GRANT app_runtime_grp TO app_runtime_user;
```

The database-level privileges were considered separately from schema and table privileges. A role may need permission to connect to a database, use a schema, and perform selected operations on approved tables. A role should not automatically receive superuser, database-creation, or role-management privileges merely because it needs to use the application.

5.3 Granting privileges according to least privilege

After the roles were created, the group assigned only the privileges required by each role. An administrator account may need broader management access, while an application-runtime account should receive only the operations required by normal application functions. A read-only account should generally be limited to SELECT access on approved tables or views. Sequence privileges should also be reviewed when tables use generated identifiers and the application needs to insert new records.

```
-- Example privilege pattern; adjust object and role names.
GRANT CONNECT ON DATABASE project_database TO app_runtime_grp;
GRANT USAGE ON SCHEMA public TO app_runtime_grp;
GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA public TO app_runtime_grp;

GRANT CONNECT ON DATABASE project_database TO app_readonly_grp;
GRANT USAGE ON SCHEMA public TO app_readonly_grp;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly_grp;
```

These statements are examples of the privilege pattern described by the activity. The actual database name, role names, tables, and privilege scope must match the project being configured. If future tables

are expected to be created, default privileges should be reviewed as a separate step so that newly created objects receive only the intended permissions.

5.4 Verifying permitted and denied operations

The activity required more than creating accounts. The group needed to test the effective permissions of the accounts. For each role, the test record should identify the login account used, the operation attempted, the expected result, and the actual result. An authorized user should be able to perform the operation required by the role. A read-only account, for example, should be able to run an approved SELECT query but should be denied an UPDATE or DELETE attempt.

The group also needed to check that ordinary application accounts did not have unnecessary database-creation or role-management privileges. If a command failed because a role or database name did not exist, the correct troubleshooting process was to inspect the actual PostgreSQL roles and databases, compare them with the SQL script, correct the mismatch or setup order, and rerun the affected step. Permission errors were to be investigated rather than bypassed by switching to a superuser account.

5.5 Documentation, repository, and handoff

The activity deliverables included the access-control plan, SQL scripts for roles and grants, privilege-verification evidence, security documentation, and an updated Activity 4 Log in the class tracker. The scripts and evidence were to be stored in the group's own repository, organized by project. Feedback about setup problems or security recommendations was to be sent to the owning groups without modifying their repositories.

The output of Activity 4 was the documented role structure and the results of permission testing. Those details became important in Activity 5 because monitoring and maintenance tasks had to be performed using accounts with appropriate privileges. A monitoring account might be able to inspect some views but not every server-level detail; such limitations needed to be recorded rather than bypassed.

6. ACTIVITY 5: DATABASE MAINTENANCE AND MONITORING

Reporting period: October 30–November 2, 2026

6.1 Identifying the databases and connecting with the correct account

Activity 5 continued with the databases prepared during Activity 4. The group used the class tracker to identify the assigned projects and database names. Before performing checks, the group selected an account whose Activity 4 role was appropriate for inspection or maintenance. Recording the account and role was important because PostgreSQL may restrict access to monitoring views or maintenance operations depending on the permissions assigned.

The first checks established which PostgreSQL server and database were being examined. The group was expected to record the PostgreSQL version, current database, database size, number of connections, and visible session activity. The following statements are examples supplied by the activity instructions and should be run against the actual database being inspected.

```
SELECT version();
SELECT current_database();
SELECT pg_size_pretty(pg_database_size(current_database()));

SELECT datname, numbackends
FROM pg_stat_database
WHERE datname = current_database();

SELECT pid, usename, state, wait_event_type, wait_event,
       query_start, query
```

```
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY query_start NULLS LAST;
```

6.2 Reviewing PostgreSQL logs

After the initial health checks, the group reviewed the PostgreSQL logs available in the configured environment. The activity instructions noted that the log location depends on the installation and server configuration. The review focused on failed login or connection attempts, SQL errors, application-generated database errors, long-running queries or blocked activity when visible, repeated warnings, and permission-denied messages.

Each finding was supposed to be supported by a log excerpt or screenshot. The report needed to explain what was observed, a plausible reason, the possible effect on the project, and a recommended next step. If no relevant issue appeared in the logs that were available, the group should state what was checked and identify any access limitations. The report should not claim that a database had no issues if the available account could not inspect the relevant logs.

6.3 Performing routine maintenance

The next part of the activity involved reviewing table and index objects and deciding whether maintenance was appropriate. ANALYZE updates the statistics used by the PostgreSQL query planner. These statistics help PostgreSQL estimate how many rows a query may return and choose an execution plan. VACUUM helps reclaim space occupied by dead row versions so that the database can continue operating efficiently.

```
ANALYZE;

-- Replace the example table with a real table in the project database.
VACUUM (VERBOSE, ANALYZE) public.example_table;
```

The VACUUM command must be run as a standalone command and not inside a transaction block. The activity instructions specifically caution against using VACUUM FULL unless it has been approved because it can require an exclusive lock and additional disk space. The account used must have sufficient privileges. If the account cannot perform the maintenance, the limitation should be recorded and a suitable maintenance role recommended rather than elevating the account without justification.

6.4 Examining query performance

For query analysis, the group selected a meaningful SELECT statement used by the application. The query was examined with EXPLAIN to display the execution plan. The plan can show whether PostgreSQL expects to use a sequential scan or an index scan and how many rows it estimates will be processed. EXPLAIN ANALYZE can provide actual execution information, but it runs the query, so it should only be used with a safe SELECT statement in the test environment.

```
EXPLAIN
SELECT *
FROM public.example_table
WHERE example_column = 'sample_value';

-- Optional: only for a safe SELECT query in the test environment.
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM public.example_table
WHERE example_column = 'sample_value';
```

A sequential scan is not automatically a problem. The group needed to consider the table size, filter condition, estimated or actual rows, and execution time before recommending an index or query change. Adding an index without a reason can increase storage use and add work to INSERT, UPDATE, and

DELETE operations. Any proposed improvement should therefore be justified by the query plan and the needs of the application.

6.5 Recording findings and preparing recommendations

The activity report for each project was expected to identify the environment and database inspected, the account and role used, the health checks performed, log findings, maintenance commands and their outcomes, the selected query and its execution plan, and any limitations. Recommendations were to be prioritized as immediate action, follow-up action, or no action required. The group also needed to organize the evidence in its own repository and update the Activity 5 Log in the class tracker.

The information collected in Activity 5 provided a baseline for Activity 6. Knowing the database size, tables, important records, constraints, access-control configuration, and any observed issues helped determine what should be included in a backup and what needed to be verified after restoration.

7. ACTIVITY 6: DATABASE BACKUP, RESTORATION, AND RECOVERY

Reporting period: November 2–6, 2026

7.1 Preparing a backup and recovery plan

Activity 6 focused on ensuring that database information could be recovered if data were lost or a database became unusable. The group first needed to identify the database to protect, the backup method, where the backup would be stored, who could access it, and how the backup would be verified. The plan also needed a proposed schedule and retention approach for a real deployment.

The plan included two recovery measurements. Recovery Point Objective (RPO) describes the maximum acceptable amount of data loss measured in time. Recovery Time Objective (RTO) describes the target time required to restore the service. The activity also required the group to explain how the Activity 4 roles, memberships, and privileges would be recreated. PostgreSQL roles are cluster-level objects and are not contained in a single-database dump, so role recreation must be planned separately.

7.2 Creating a logical backup

The activity instructions specified a logical backup using pg_dump in custom archive format. The -Fc option creates a custom-format archive that can later be inspected and restored with pg_restore. The command below is the activity template; the placeholders must be replaced with the actual account, database name, and output file path when the command is run.

```
pg_dump -U <username> -h localhost -p 5432 -Fc \
  -d <database_name> -f <database_name>_activity6.dump
```

The group was expected to record the command used, backup file name, file size, checksum, date and time, and any errors. A password should not be placed directly in the command or committed to the repository. The backup file should be kept in a protected local location and should not be uploaded to a public repository because it may contain project data.

7.3 Inspecting the backup archive

After creating the archive, the next step was to inspect its contents with pg_restore --list. This command lists the database objects recorded in the archive. The listing helps confirm that the archive contains expected objects, but the presence of a file and a successful listing do not prove that the database can be restored. A separate restore test is necessary to verify that the archive is usable.

```
pg_restore --list <database_name>_activity6.dump
```

7.4 Restoring to a separate test database

The activity required the group to create a new, empty test database with a distinct name. The backup was to be restored into this test database rather than overwriting the original project database. This protects the original environment while allowing the group to verify the backup.

```
createdb -U <admin_username> -h localhost -p 5432 <restore_test_database>

pg_restore -U <admin_username> -h localhost -p 5432 \
  -d <restore_test_database> <database_name>_activity6.dump
```

If ownership or privilege settings in the archive did not match the test environment, the group needed to investigate the warning and use an instructor-approved restore approach. Options that suppress errors should not be added blindly. Any warning, failure, or difference should be documented with the reason and corrective action.

7.5 Verifying the restored database

After restoration, the group needed to compare the restored database with the expected project structure. The verification included checking that the expected tables and important objects existed, selected sample records were present, primary and foreign keys and other constraints were retained, and a representative SELECT query returned the expected results. The Activity 4 roles, memberships, and privileges also needed to be recreated or checked, followed by one permitted and one denied operation. If feasible, the application could also be configured to connect to the restored test database.

7.6 Conducting a controlled recovery simulation

The recovery simulation was designed to use only disposable test data in a restored test database. The group first recorded the expected state of a small test record. It then simulated a limited incident, such as deleting that test record, and recovered the expected state by restoring the backup into a fresh recovery database or by using a carefully planned table-level recovery method. The recovered data was compared with the original expected state.

The original database and the initial backup were to remain untouched throughout the simulation. The activity instructions explicitly prohibit destructive testing on a production database or on the only copy of a project. The report should explain the simulated incident, the state before and after the loss, the recovery method, whether the data was recovered, and what could be lost if the incident occurred after the most recent backup.

7.7 Evaluating recovery readiness

The final evaluation considered whether the backup method and proposed frequency would be sufficient for a real parking-management system. A system that records vehicle entry, exit, and payment information may require a shorter backup interval than a project with infrequent changes. The group should consider how much data could be lost between backups, how quickly the service needs to return, who is authorized to access backup files, and how long backups should be retained.

The most important principle of this activity is that a backup is not proven reliable until it has been restored and checked. The group's report should therefore distinguish between a backup that was created, an archive that was inspected, a database that was restored, and a recovery simulation that was completed. Each stage needs its own evidence.

8. ISSUES ENCOUNTERED AND TROUBLESHOOTING

8.1 Issues recorded in the original Park It development report

- The initial interface was static. The documented response was to add CRUD functionality so the application could manage database records rather than only display a screen.
- Node.js and Git commands were not recognized in the terminal. The documented response was to check and correct the installation paths, then restart the development environment.
- A Git merge conflict occurred during the final push. The documented response was to resolve the conflict by updating the main branch using the working local copy.

8.2 Issues to record during Activities 4–6

The later activity reports should describe only errors that actually occurred and should include the exact message, affected database or script, correction, and retest result. The following are troubleshooting situations described by the activity requirements, not claims that every situation occurred in the group's work:

- A role or database name in a SQL script does not match an existing PostgreSQL object. Check the actual names, correct the script or creation order, and rerun the affected statement.
- A permission-denied error occurs during access testing or maintenance. Compare the requested operation with the access-control plan and record whether the denial is expected or whether a required privilege is missing.
- Monitoring views or server logs are not fully visible to the selected account. Record the limitation and use an appropriately authorized account only when permitted.
- A backup restore reports ownership or privilege differences. Inspect the warning, check the target database and archive contents, and document the approved restore options instead of suppressing errors without investigation.
- A query plan shows a sequential scan. Consider table size and filter selectivity before recommending an index; a sequential scan may be reasonable for a small table.

8.3 Progress statements that need reconciliation

The original Activity 3 report lists database connection as completed at 80%, while the development log describes the connection implementation as completed. It also lists CRUD implementation as ongoing at 60%, while the CRUD test section marks all four CRUD operations completed. These differences should be reviewed using the actual commit history and dated screenshots. The final progress report should explain whether the percentage refers to partial module coverage, implementation completion, or testing completion.

9. EVIDENCE AND DOCUMENTATION STATUS

The screenshots in this version were recovered from the original 13-page Park It project documentation PDF. They are the original project images, not newly generated illustrations. They support the Activity 3 sections on database design, SQL schema, user interface, and CRUD testing.

Evidence included in this document	Source / coverage
ERD screenshot	Original project documentation; Activity 3 database design.
SQL schema screenshot	Original project documentation; Activity 3 database implementation.
Login and dashboard screenshots	Original project documentation; Activity 3 application interface.

Vehicle, parking monitoring, and payment screenshots	Original project documentation; Activity 3 application modules.
Create, retrieve, update, and delete screenshots	Original project documentation; Activity 3 CRUD testing.
Activity 4–6 command output and screenshots	Not present in the original project PDF; attach the group's actual screenshots or terminal output before claiming the related test was completed.

For Activities 4–6, the detailed steps and command examples in this document follow the course activity requirements. The original Park It PDF supplied for this revision did not contain the group's Activity 4, Activity 5, or Activity 6 screenshots, command outputs, backup archive listing, restore verification, or recovery-simulation evidence. Those results should be added from the group's actual work. This avoids presenting an instructed procedure as if it were verified output.

10. REPOSITORY AND EVIDENCE ORGANIZATION

The group should continue using its own repository and keep the evidence organized by activity. A possible structure is shown below; folder names may be adjusted to match the existing repository.

```
PARK-IT-DATABASE-MANAGEMENT-SYSTEM/
|-- README.md
|-- docs/
|   |-- progress-documentation.pdf
|   |-- project-development-documentation.pdf
|   `-- access-control-plan.md
|-- database/
|   |-- schema/
|   `-- seed-data/
|-- db-access-control/
|   |-- role-creation.sql
|   |-- privilege-grants.sql
|   `-- verification/
|-- db-maintenance/
|   |-- health-checks/
|   |-- logs/
|   `-- query-plans/
`-- db-backup-recovery/
    |-- backup-plan.md
    |-- archive-inspection/
    |-- restore-verification/
    `-- recovery-test/
```

Do not commit real passwords, API keys, or database backup files containing project data. Use placeholders in documentation and scripts, and keep actual credentials and dump files in protected local storage. Screenshots should be reviewed before submission to ensure that they do not expose passwords or other secrets.

11. AI USAGE DECLARATION

AI tools were used to assist with organizing the documentation, improving grammar and clarity, explaining database-administration concepts, and preparing example SQL command formats. The original screenshots and project-specific details in this report were retained from the group's earlier documentation. AI-generated explanations and command examples must be checked against the course

instructions, the actual PostgreSQL environment, and the group's command results. The group remains responsible for verifying that every reported task and outcome matches the work actually performed.

12. CONCLUSION

The Park It project documentation shows the progression from planning and database design to application implementation and CRUD testing. The original project evidence demonstrates the ERD, SQL schema, login page, dashboard, parking-related modules, and record-management tests. The succeeding database-administration activities build on this foundation. Access control establishes which accounts may perform database operations, monitoring and maintenance provide information about database condition and query behavior, and backup and recovery procedures help protect the stored records.

The activities are connected because each stage prepares information needed by the next. The database objects and application roles identified during development support access-control planning. The access-control configuration determines which accounts can perform monitoring and maintenance. The monitoring results then help identify what must be included and verified in a backup. Before submission, the group should attach its actual Activity 4–6 evidence and reconcile the progress percentages that differ in the original Activity 3 report.